Supplementary Document: Author Contributions and Scope Differentiation

Companion to "Drone-Guided Autonomous Navigation: A Cooperative UAV–AMR System for GPS-Denied Environments"

D. Ortiz A. Romo N. Rosales S. Pulido J. González

Tecnológico de Monterrey — 2026

This document accompanies the final report. It records (i) which authors contributed to each module and submodule, (ii) the parts of the delivered system that go beyond the basic project specification, and (iii) an individual conclusion and reflection from each author.

1 Author Contributions Matrix

Mark **X** in the cell of every author who contributed to a given module or submodule. Authors: **D**avid Ortiz, **A**lfredo Romo, **N**oé Rosales, **S**ebastián Pulido, **J**orge González.

Module / Submodule	**David**	**Alfredo**	**Noé**	**Sebastián**	**Jorge**
A. UAV Flight & Survey					
Tello ROS driver (`tello_driver`)			x		
Position controller + feedforward (`tello_pos_control`)			x		
Boustrophedon scan-trajectory generator			x		
OptiTrack fault detector (alternating-ghost)					x
B. Aerial Mapping (`arena_map_builder`)					
Image-stitching pipeline (similarity transform, RANSAC)	x	x			
Feature front-end — SIFT	x				
Feature front-end — SuperPoint (ONNX/GPU)	x				
Laplacian-pyramid blending & canvas management	x				
Obstacle transfer & consistency scoring	x				
Occupancy rasterization (confidence halos)	x			x	
Map-quality RandomForest classifier (+ training)	x				

continued on next page

Module / Submodule	David	Alfredo	Noé	Sebastián	Jorge
C. Aerial Marker Localization (`arena_marker_localizer`)					
ArUco IPPE PnP + transform chain	x			x	
Robust aggregation (Weiszfeld geometric median)	x				
Bias calibration (IRLS, cross-validation)	x				x
D. AMR State Estimation & Localization					
Custom EKF (`ekf_amr`) — process/measurement models			x		x
Adaptive VIO gate (`ema_gate`, burst detection)	x		x		
cuVSLAM (Isaac ROS) integration	x		x		
ArUco wall-marker localizer (`aruco_localizer`)				x	
Multi-marker fusion + distance-linear covariance				x	
world→odom alignment (`alignment_node`)				x	
E. AMR Drive Stack (`amr_bringup`)					
Hardware driver, IMU remap, e-stop hook (`driver_node`)			x		
Wheel odometry (`wheel_odometry_node`)			x		
Feedback-linearization controller (`controller_node`)			x		
F. Navigation & Planning					
Bayesian occupancy mapping (`world_mapper`)				x	
Map fusion (`fusion`)				x	
A* planner + spline follower (`trajectory_planner`)			x		x
Frontier-exploration fallback — FSM controller					x
Image-space visual-servo homing					x
G. Physical Safety					
Automatic emergency braking (`emergency_stop`)					x
H. Network Security (`ros2_security`)					

continued on next page

Module / Submodule	David	Alfredo	Noé	Sebastián	Jorge
SecureEnvelope crypto (RSA-PSS, AES-256-GCM)		x			
PKI / certificate generation tooling		x			
Policy, mixin, mixed-trust, kill switch		x			
Performance benchmark suite		x			
I. Orchestration & Integration					
Mission orchestrator FSM (`mission_orchestrator`)	x				
Graceful degradation & safe-abort handling	x			x	
Multi-host networking (DDS, SSH control plane)	x	x		x	
J. Documentation & Evaluation					
Final technical report	x	x	x	x	x
Per-subsystem documentation (`DOCS.md`)	x	x	x	x	x
Experimental evaluation & data collection	x	x	x	x	x

2 Differentiation from the Basic Specification

The challenge specification defines nine *required* features (85/100) and five *extra* features (15/100). This section lists the parts of the delivered system that go beyond the required baseline, mapping them to the official extra list where applicable.

2.1 Extras claimed from the official extra-feature list (15/100)

Official extra #1 (*dynamic speed adjustment*) is **not** claimed as an extra here: it is delivered as part of the required Stage-4 robustness demonstration (arc-length trajectories with a trapezoidal speed profile). The remaining official extras are claimed as follows:

- **(E2) Embedded Deep-Learning vision.** SuperPoint deep features exported to ONNX and run GPU-accelerated on the Jetson Orin Nano (an alternative to the SIFT front-end).
- **(E3) Quantitative improvement metrics.** EKF position-error reduction vs. wheel odometry, UAV tracking-error statistics, and a full cryptographic latency/throughput benchmark of the security layer.
- **(E4) Robustness to perturbations / noise.** Adaptive VIO gate against VSLAM loop-closure spikes; OptiTrack alternating-ghost fault detector; replay/tamper/downgrade defenses; graceful comms degradation; and graceful degradation in the mission orchestrator (frontier fall-back on map rejection, drone-safe abort on any sub-stage failure).

Official extra #5 (*documented industrial standards*) was **not** pursued and is not claimed.

2.2 Additional engineering beyond the required scope

These were not explicitly requested by the specification:

- **cuVSLAM (Isaac ROS) integration** as the visual–inertial odometry source feeding the EKF — an additional capability beyond the required cooperative localization.
- **Learned map-quality classifier** (RandomForest) gating the aerial map, with a safe-fail sentinel. The model also contributes to the required Deep-Learning-for-vision feature (R6); its use as a *runtime quality gate* is the extra contribution.
- **Frontier-exploration fallback** that autonomously recovers the mission when the aerial map is rejected (FSM + frontier scoring + visual-servo homing).
- **Graceful degradation in the mission orchestrator**: classifier-gated fallback split and a drone-safe abort sequence on failure.
- **Security performance benchmark and threat model** (counted under E3) beyond simply "implementing security."
- **Robust marker localization**: Weiszfeld geometric-median aggregation and an IRLS bias-calibration model.
- **Similarity-transform stitching** with consistency scoring and confidence-weighted occupancy halos (chosen over full homography for stability over the repetitive grid).
- **Application-layer SecureEnvelope** with a global kill switch, mixed-trust bridging.

3 Individual Conclusions and Reflections

Each author writes their own conclusion and reflection below (contributions, challenges, learnings).

David Ortiz

My work centered on the aerial perception stack — the image-stitching pipeline that turns drone video into a metric map and the marker localization that anchors the goal — and on the mission orchestrator that ties the whole system together. If one theme runs through all of it, it is the discipline of choosing the right tool for each problem rather than the most fashionable one. It is tempting to reach for a deep network at every step, but much of this system earned its reliability precisely by *not* doing so: a 4-DoF similarity transform instead of a full homography, because the extra freedom only invited drift over the arena's repetitive grid; a RandomForest rather than a neural net to judge map quality, because it was interpretable, tiny, and ran in pure NumPy on the Jetson. Deciding where deep learning genuinely helped and where classical geometry and a well-chosen heuristic were faster and more robust proved a harder, and more valuable, skill than implementing any single method.

The stitching pipeline was my biggest personally built technical challenge. Building a fast, non-deep-learning computer-vision pipeline that yields a robust, fault-tolerant map sounds simple until the repetitive blue-tape grid starts matching the wrong cells and a single bad frame fans the entire canvas into nonsense. Getting it right meant layering defenses the rest of the team rarely saw: HSV feature-exclusion masks, MAD pre-filtering before RANSAC, consistency scoring across independent projections, confidence-weighted occupancy, and a learned quality gate as the final

safety net. Watching years of computer-vision coursework turn into a pipeline that holds together over hundreds of frames, on real hardware, was the most satisfying engineering of the project.

The orchestrator was the opposite kind of challenge, and just as rewarding. Once every subsystem worked in isolation, the last layer of autonomy was making them cooperate from a single command — one click that brings up OptiTrack, flies the survey, builds and grades the map, hands it off, and drives the AMR to the goal, aborting safely if anything fails. Designing something simple enough to reason about yet powerful enough to coordinate three networked machines taught me that the real difficulty of robotics is rarely any one algorithm; it is the integration — the frames, the timing, the failure modes, and the hand-offs between parts built by different people. Bringing a full robotic system together, end to end, is a skill in its own right, and I hope this project sets the basis for a next step in my professional path in robotics.

Alfredo Romo

My primary contribution to the project was the cybersecurity layer — the application-level secure middleware that authenticates and optionally encrypts the ROS 2 traffic between the drone, the Jetson, and the ground robot. Before settling there, I also took an early attempt at the aerial mapping: stitching the drone's frames using its onboard odometry to place each image on the canvas, rather than matching image features. In principle the drone's pose should have told us exactly where each frame belonged; in practice the odometry was too noisy and drifted enough that the mosaic never held together, and the team moved to the feature-based pipeline instead. It was a useful dead end — it made clear why the vision-driven approach was worth the extra complexity.

The cryptographic design was the most intellectually demanding part. Choosing to derive the AES-256 session key directly from a sender's public certificate — rather than negotiating a shared secret — solved a real distribution problem cleanly. The trade-off is that confidentiality is only as strong as certificate control, not forward secrecy; but for a robotics system whose threat model is spoofing and tampering on a local network rather than long-term key compromise, that was an acceptable trade. The decision to verify before decrypt — rejecting unverified ciphertext before touching it — was a subtle but important defense against decrypt-oracle attacks, the kind of detail that is easy to get wrong the first time.

Not every difficulty was conceptual. ROS 2 Foxy backwards compatibility surfaced how much the Python API had shifted between distributions — small differences in lifecycle, timer behavior, and message serialization that only appear at runtime and are not well documented. The main learning, though, was about adoption: in security-adjacent work, the clearest wins come from making the secure path the path of least resistance. The global kill switch, the per-node profiles, and the opt-in decorator pattern all push in that direction — security fails not when the cryptography is weak, but when "doing it right" costs too much for anyone to bother.

Noé Rosales

My responsibilities spanned the UAV position controller and the AMR state estimator and trajectory-tracking controller. Working across both ends of the cooperative architecture gave me a complete

view of where the system's fragility truly lived and it was rarely where I expected.

The deepest challenge for me was the EKF architecture. The hardest lesson was conceptual, treating wheel velocity as a control input rather than a measurement overwrites the filter's state on every cycle and renders every other sensor irrelevant. Recognizing that wheels, IMU, and visual odometry must all enter as measurements, with their own observation models and noise matrices, required stepping back from the implementation entirely. That reframing, more than any parameter tuning, produced a filter that behaved correctly. The cuVSLAM integration compounded the difficulty: loop-closure spikes reaching nearly 20× the robot's true cruise velocity exposed the limits of standard filtering approaches, and the adaptive EMA gate emerged not from a textbook but from repeatedly asking what the spike actually was physically, and designing a rejection rule to match that behavior.

Against that background, the first clean end-to-end run drone survey, map handoff, AMR navigating autonomously to the goal felt genuinely earned. The broader takeaway is that real robotic systems are integration problems as much as algorithmic ones. Each subsystem documented in this report is individually tractable; making them work together, reliably, across three networked platforms and under hard timing constraints, is where the actual engineering happens.

Sebastián Pulido

My primary contribution to the project was the localization and mapping pipeline for the AMR within the arena. The first approach was to use SLAM Toolbox to generate a local map; however, since the intent was to fuse the drone's top-down aerial map with the AMR's local map, both maps needed to be spatially aligned. Because the navigation goal resides in the inertial "world" frame, we decided to align the local map with the aerial map — which is already expressed in the world frame — and plan over the fused result. Initially, we treated the problem as a pure image-alignment task with no additional information, applying Iterative Closest Point (ICP) and a coarse search to find the transformation between the two maps. This did not yield satisfactory results. A second attempt consisted of directly applying the world → slam_map transform to the AMR map. This relied on the assumption that the SLAM frame starts aligned with the odometry frame – which it does, but only initially. As scan matching and loop closure corrections accumulate, SLAM Toolbox continuously modifies the slam_map → odom transform, shifting the odometry frame so that the trajectory integrated by the Extended Kalman Filter (EKF) stays consistent with the corrected poses produced by the SLAM backend. These adjustments proved to be insufficiently deterministic to correct in a time-efficient manner, so we decided to abandon the SLAM-based approach altogether. The adopted solution was to define a fixed occupancy grid sharing the same origin, orientation, dimensions, and resolution as the drone map. With both grids spatially identical, map fusion reduces to a trivial cell-by-cell operation. To project LiDAR scans onto this fixed grid, the system performs a TF lookup for world → base_footprint, aided by the dynamic tf world->odom which is published by a dedicated node called alignment_node. The remaining challenge is that part of this transform chain — specifically odom → base_footprint — is subject to odometric drift. To address this, I developed the aruco_localizer node, which uses fixed ArUco markers placed throughout the arena as absolute position references, continuously correcting the AMR's estimated pose.

Jorge González

During the development of this project, I chose to get involved in a variety of tasks with the goal of learning from my teammates and strengthening my skills in areas I had not previously explored in depth. My primary contributions were the EKF implementation, initially a fusion of IMU data and wheel encoder odometry, the trajectory planner, and the frontier-based exploration stack. We faced several significant challenges throughout the implementation process. The most persistent was sensor noise: the inherent imprecision of real hardware repeatedly exposed edge cases that had not been considered during the design phase, causing algorithms that worked perfectly in simulation to behave unexpectedly on the physical robot. Hardware failures added another layer of difficulty, multiple Yahboom motors burned out over the course of the project, introducing delays that forced us to reorganize our schedule and adapt. Perhaps the most demanding phase, however, was system integration. Connecting all the modules together always requires meticulous care, because a mismatch between the data format one node produces and what another expects can produce failures that are extraordinarily difficult to trace. Units, coordinate frames, QoS policies, and timing assumptions all became sources of subtle bugs that consumed significant debugging time. Finally, this project deepened my appreciation for ROS 2 as a framework for building complex, multi-module systems. The ability to develop, test, and replace individual nodes independently, while relying on a common communication layer for topics, transforms, and parameters, made collaboration across a large codebase genuinely manageable. What could have been an integration nightmare was instead a structured process, and that structure came almost entirely from ROS. I have a much clearer sense of how to design node interfaces from the start with integration in mind, rather than treating it as an afterthought.